# Recordings, Not Rows: Lossless Waveform Storage and Lag-Tolerant Retrieval Inside PostgreSQL

Working draft 0.2 — not for distribution

Author(s) to be confirmed
Affiliation to be confirmed

## ABSTRACT

Vibration, seismic and physiological archives hold uniformly sampled recordings, yet relational systems store them as rows of samples or as detached feature vectors. We describe MorletDB, a PostgreSQL extension that stores each recording as losslessly coded chunks with transactional, replay-safe appends, keeps search representations as replaceable materialized views of those chunks, and returns search hits as sample intervals that read back bit-exactly. We evaluate it with frozen protocols on six real sensor inputs and on fresh seismic data. Three findings generalize beyond the prototype. First, sensor floats are usually calibrated integers, and decimal-oriented float codecs miss this: on two calibrated inputs ALP falls back to about 60 bits per sample, while grid recovery with last-place corrections needs 9.2–15.5, 1.5–1.6× less than the best metadata-free codec tested; the remaining gap to integer codecs is entirely correction cost. Second, storing the same envelopes in pgvector costs identical space, but zero-lag cosine search shares only 65% of the lag-tolerant top ten: lag tolerance changes the query, not just its speed. Third, on 235 thousand fresh seismic windows, exact in-database lag-tolerant search costs a predictable 12 µs per window, and an HNSW index on the same envelopes is not a usable prefilter: re-ranking its top 1000 recovers only 74% of the exact top ten. We also report the negative results that shaped the design: wavelet and progressive codecs, cross-recording reference coding and a Morlet transform all failed to beat simpler alternatives.

## 1 INTRODUCTION

Condition monitoring, seismology and clinical physiology produce long, uniformly sampled recordings. An analyst first finds an interval of interest, then asks where similar behaviour occurred and what the original samples show. That workflow needs three things at once: compact storage of the original samples, a search representation, and a reliable link from every hit back to its samples and acquisition context.

Relational systems offer two layouts, each covering part of the problem. One row per sample makes every value addressable but costs about 93 bytes per sample uncompressed in our measurements; compressed time-series extensions shrink this but treat samples as independent floats. A vector per window gives a compact search key, but the application must maintain the link to the source samples, keep representations compatible, and decide what similarity means. Neither layout captures a *recording*: a channel with a sample rate, a unit, a calibration and a half-open sample coordinate system.

MorletDB is a PostgreSQL extension (Rust, via pgrx) built around that unit. Recordings are appended in chunks at global sample offsets; raw chunks, derived representations and checkpoints commit in one transaction; and every search result is a recording identifier plus a sample interval that reads back through the same extension. The design rests on one premise: *authoritative samples and search representations should be stored separately, but connected by explicit interval semantics and transactional ingestion.* The name reflects the first representation tried, Morlet wavelet energies; our experiments later showed no advantage over FFT band powers (Section 7), and the representation is now a versioned, replaceable part.

This paper makes four contributions.

(1) **A recording-oriented design inside PostgreSQL**: replay-safe chunk appends at global offsets, lossless chunk coding, materialized and replaceable search representations, and interval read-back, under PostgreSQL transactions and permissions (Section 3).
(2) **A storage characterization of calibrated sensor floats** on identical samples against ALP, Parquet, zstd, FLAC and DuckDB (Section 5). Calibrated floats defeat decimal-oriented float codecs; recovering the digitizer grid closes most of the gap to integer codecs; the remaining gap is the cost of reproducing the source's floating-point rounding.
(3) **An analysis of lag-tolerant retrieval in a relational engine**: precision of materialized envelopes, equal storage but unequal queries against pgvector, a scaling study on fresh data showing that exact lag-tolerant scans are linear and that a standard vector index is not a sufficient prefilter for them (Section 6).
(4) **Negative results that shaped the system**, each from a frozen protocol (Section 7).

We do not claim a new entropy coder, a new similarity measure or a sublinear lag-tolerant index. Code, protocols, raw results and the commit at which each protocol was frozen are public [27].

## 2 DATA MODEL AND SEMANTICS

A *recording* is a sequence $x_0, \ldots, x_{N-1}$ of float64 values with a sample rate $f_s$, a channel, a unit, a declared sample quantum $q$ (Section 3.2) and JSON acquisition metadata. An *interval* $[a, b)$ denotes $x_a, \ldots, x_{b-1}$. Sample offsets are authoritative; wall-clock time additionally needs a time origin, and a real acquisition gap starts a new recording rather than silently joining samples.

A search representation $\phi_v(x_{a:b})$ is identified by a versioned extractor name $v$. Two windows are comparable only if their recordings share sample rate, channel and unit and their windows share length and extractor; metadata containment filters restrict candidates further. These rules reject comparisons that are computable

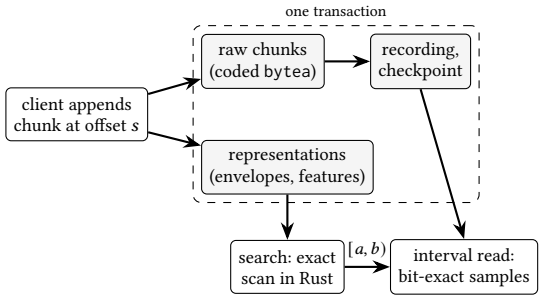



**Figure 1: Write and read paths of a collection. Raw chunks, derived representations and the checkpoint commit together; search returns intervals, which are read back from the raw chunks, not from the representation.**

but meaningless, such as distances between features from different sample rates.

We separate three notions of correctness that are often conflated. *Sample identity*: decoding returns the appended float64 bit patterns. *Search fidelity*: a stored or approximate representation reproduces a reference ranking, measured here by recall@10. *Task relevance*: retrieved intervals help answer a domain question. Each needs its own evidence; fidelity says nothing about relevance. We call results *source-linked* in the operational sense that each hit carries its recording, interval and metadata; this is not cryptographic provenance.

# 3 SYSTEM DESIGN

## 3.1 Collections and transactional appends

A collection is a set of caller-owned tables created by `morlet_create_stream_collection`: recordings with their checkpoint, raw chunks, and per-representation tables (Figure 1). All functions run as `SECURITYINVOKER`, so PostgreSQL privileges and row-level security apply to every read and search. An append names its global start offset:

```
SELECT morlet_start_recording('signals', 'pump7', 25600,
  'accel_x', 'g', '{"asset":"P7"}', 512, 256, 'zscore',
  'none', 0.001);                          -- quantum q
SELECT morlet_append_chunk('signals', 'pump7', 0, :x);
SELECT * FROM morlet_search_envelope_interval('signals',
  'pump7', 81920, 86016, '{"asset":"P7"}', 10);
SELECT samples FROM morlet_read_interval('signals',
  'pump7', 81920, 86016);
```

Raw samples, derived representation rows and the checkpoint commit in one transaction, so a committed checkpoint always refers to committed samples. An append that repeats a committed range with identical samples is verified and accepted as a no-op; a changed retry, a gap or an overlap fails. Collectors can therefore retry after a crash without their own deduplication. Windows of derived representations may cross chunk boundaries.

## 3.2 Lossless chunk coding

Each raw chunk is one `bytea` value. The codec combines established techniques: fixed polynomial predictors of order 0–4 as in Shorten and FLAC [5, 6], partitioned Rice coding of zig-zag mapped residuals [7], and a CRC-32C checksum. Its distinctive part is how it treats floats.

Most sensor values are calibrated integers: a digitizer emits counts $d$, and acquisition software stores $x = f(d)$, for example $x = d \cdot s$ (seismic counts, WAV) or, in EDF [8], $x = (d - d_{\min})(p_{\max} - p_{\min})/(d_{\max} - d_{\min}) + p_{\min}$. The recording declares a nominal quantum $q$: 1 for counts, 0.001 for three decimals, the ADC step for calibrated values. Per chunk, the encoder seeks a grid

$$x_i = o + k_i\, s \oplus \delta_i, \tag{1}$$

with integer indices $k_i$, a step $s$ refined from $q$ to within one unit in the last place (ulp), an offset $o$ (format version 3; zero in version 2), and corrections $\delta_i$, stored as differences of order-preserving integer images of float64, so that decoding reproduces every bit pattern. A division mode $x = k/d$ covers decimal steps exactly. Indices are predicted and Rice-coded like integers; corrections form a second stream. If no grid fits within a thousandth of a step, the chunk is coded as order-preserving float bits, and if that is not smaller, as raw float64. The quantum is a storage hint, never a conversion: a wrong quantum costs space, not accuracy.

## 3.3 Materialized, replaceable search representations

Search representations are rows derived from raw chunks and keyed by recording, window size and start sample. We study the envelope representation: for a 4096-sample window, z-score, take the magnitude of the analytic signal, smooth with a 50-sample moving mean, keep every tenth value, and z-score again. This gives 410 values, stored as float16 (821 bytes with a version byte). The distance between a query envelope $q$ (full precision) and a stored envelope $g$ is

$$D(q,g) = 1 - \max_{|\ell| \le L} \frac{1}{n} \sum_t q_{t+\ell}\, g_t, \qquad L = \lfloor 0.075\, n \rfloor = 30, \tag{2}$$

a biased cross-correlation over bounded shifts. Search is an exhaustive scan of eligible envelopes scored in Rust (`morlet_envelope_distance`, eight lags per inner block), with the source interval excluded and overlapping windows of one recording suppressed greedily in rank order. Materialization and float16 storage change values relative to envelopes computed from raw samples; Section 6 measures the effect. Dropping and rebuilding a representation never touches raw chunks.

# 4 EVALUATION METHOD

Every study follows the same discipline: a protocol with questions, data, arms, metrics and decision thresholds is committed before data is read or measured; harness and tests are committed before the run; each study runs once; deviations are logged in the protocol. Table 1 lists the studies used here and marks which used data no earlier study had read.

All runs used one Apple M5 Max (macOS, 128 GB), PostgreSQL 17.11 built by pgrx, pgvector 0.8.0 and one client over localhost. TimescaleDB 2.30.2 ran in Docker; we compare its sizes, not its speed.

**Table 1: Studies. "Fresh": data not read by any earlier study.**

| Study | Workload | Data |
|---|---|---|
| P2 | DB storage vs TimescaleDB; 4 inputs × 4.2 M samples | reused |
| B1 | codec vs ALP, Parquet, zstd, FLAC, DuckDB; 6 inputs | 2 new inputs |
| R1b | envelope precision; 1,826 windows, 1,557 queries | fresh |
| P1 | the same envelopes in pgvector | R1b's |
| R2 | search scaling to 236 k windows; HNSW prefilter | fresh |
| S1 | sleep staging by retrieval; 39 held-out subjects | fresh split |

**Table 2: P2: total database storage in bits per sample. T: the TimescaleDB layouts tested (a row per sample with an integer or float column; a row per 4096-sample array).**

| Input | MorletDB | T int | T float | T array |
|---|---|---|---|---|
| CX seismic counts | **6.12** | 8.25 | 14.34 | 25.0 |
| IMS three-decimal | **11.36** | 14.06 | 71.95 | 28.1 |
| Paderborn counts | **9.95** | 12.89 | 24.58 | 26.4 |
| Paderborn physical | **11.53** | — | 70.12 | 29.5 |

# 5 STORAGE

## 5.1 Database footprint (P2)

P2 appended the same 4,194,304 samples of four inputs (CX seismic counts [23], IMS bearing vibration [26], Paderborn bearing vibration [25] as ADC counts and as physical floats) to MorletDB and to TimescaleDB hypertables with native compression (Gorilla for floats; delta-of-delta and Simple-8b for integers), and measured total on-disk bytes, including indexes, TOAST and metadata tables, after verifying that four intervals per layout read back bit-exactly [28].

TimescaleDB on integer columns comes closest but is 24–35% larger (Table 2), and requires the user to convert values to counts first. On float columns, Gorilla's XOR coding *expands* noisy decimal and calibrated values beyond 64 bits per sample. We tested one `segmentby/orderby` configuration and not TimescaleDB's newer column formats. Of MorletDB's footprint, the codec payload is 4.6–9.2 bits per sample (Table 3); PostgreSQL row and TOAST overhead adds 1.5–2.6 bits per sample at 4096-sample chunks.

## 5.2 Codec against modern baselines (B1)

B1 compares codecs on identical samples, each verified lossless [29]. Float codecs receive only float64 values: zstd level 19 per chunk, Parquet with `BYTE_STREAM_SPLIT` and zstd, and DuckDB 2.0's automatic float compression, which uses ALP and its fallback ALP-RD [1, 13]. MorletDB receives the values plus the nominal quantum. Integer codecs receive the exact digitizer counts: FLAC [6], Parquet delta coding with zstd, and DuckDB bit-packing. Two inputs are new to our storage studies: a Sleep-EDF EEG slice [20, 21] from 32 nights with symmetric and asymmetric calibration ranges, and PTB-XL ECG lead I [22].

Figure 2 and Table 3 show three regimes.

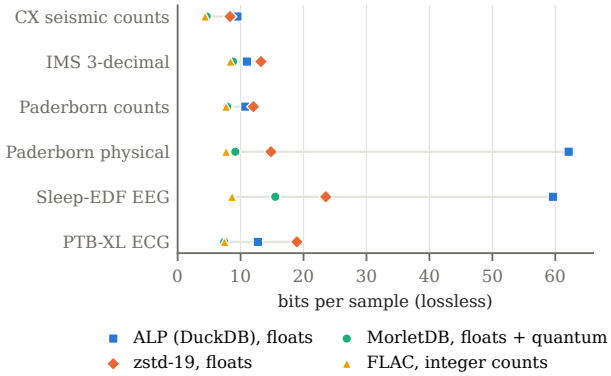



**Figure 2: B1: lossless bits per sample on identical samples. Float codecs see only float64 values; MorletDB also sees the nominal quantum; FLAC sees integer counts. Table 3 gives all arms.**

**Table 3: B1: bits per sample. Z: zstd-19; PQ: Parquet; DDB: DuckDB.**

| | float input | | | | integer input | | |
|---|---|---|---|---|---|---|---|
| Input | Z | PQ | ALP | **MorletDB** | FLAC | PQ | DDB |
| CX counts | 8.33 | 6.84 | 9.50 | **4.61** | 4.38 | 5.24 | 6.38 |
| IMS 3-decimal | 13.24 | 42.94 | 11.00 | **8.81** | 8.41 | 9.87 | 10.88 |
| Paderborn counts | 12.04 | 9.48 | 10.75 | **7.88** | 7.70 | 9.61 | 11.25 |
| Paderborn physical | 14.82 | 30.57 | 62.13 | **9.17** | 7.70 | 9.61 | 11.25 |
| Sleep-EDF EEG | 23.54 | 38.15 | 59.63 | **15.51** | 8.65 | 10.22 | 12.00 |
| PTB-XL ECG | 18.94 | 44.64 | 12.75 | **7.40** | 7.46 | 10.27 | 12.75 |

*Counts and decimals.* When values are integers or short decimals (CX, IMS, Paderborn counts, ECG with gain 1000), ALP and MorletDB both recover integers; MorletDB is 20–51% smaller than ALP because consecutive samples are correlated and prediction exploits that, while ALP's frame-of-reference bit-packing does not. MorletDB is within 5% of FLAC, whose LPC stage it lacks, and slightly below FLAC on ECG.

*Calibrated floats.* On Paderborn physical values and Sleep-EDF, no decimal exponent turns values into integers, so DuckDB chose ALP-RD and stored about 60 bits per sample, near raw. The best metadata-free codec was zstd. MorletDB with the nominal quantum was 1.5–1.6× smaller than zstd. Our pre-registered claim of at least 2× was not met.

*The correction gap.* Given counts, MorletDB codes the two calibrated inputs at 7.9 and 8.8 bits per sample; given the calibrated floats, it needs 1.3 and 6.7 bits per sample more. The difference is the correction stream. The source software computed $x$ with a particular sequence of roundings (on Sleep-EDF a multiplication, a division and an addition), which a single fitted step and offset cannot reproduce for every sample. A grid that replays the source's calibration arithmetic exactly would remove the corrections; we have not built it. Our pre-registered claim that MorletDB stays within 15% of the best integer codec held on four of six inputs and failed on these two.

**Table 4: R1b on fresh 2022 data. Size relative to the raw chunk table; latency includes the client path.**

| Envelopes | Size/raw | Recall@10 | Same order | p95 ms |
|---|---|---|---|---|
| from raw (reference) | — | 1 | 1 | 1324 |
| float32 | 28.1% | 1.000 | 1.000 | 14 |
| **float16** | **12.9%** | **0.9999** | **0.987** | **16** |
| uint8 | 7.2% | 0.998 | 0.845 | 17 |
| uint8, coarser | 4.2% | 0.952 | 0.067 | 16 |

The lesson for float compression is that "real-valued" sensor data is often neither decimal nor arbitrary: it is an affine image of integers under floating-point rounding. Decimal-oriented codecs fail on it, and generic byte-oriented ones leave most of the savings unused.

# 6 RETRIEVAL

## 6.1 Precision of materialized envelopes (R1b)

R1b selected an envelope precision on 2023 seismic windows and evaluated it once on 2022 data that no earlier study had read: 1,826 windows at ten CX stations [23], 1,557 event windows from 284 catalogued events and 269 no-match distractors [30]. The reference decodes raw samples and computes envelopes in the client.

Float16 envelopes met all frozen criteria (Table 4): recall@10 of 0.9999, 12.9% of the raw table, and an unchanged catalogue-distance relevance proxy (+0.001 km, 95% interval [−0.002, +0.004]). The roughly 80-fold latency reduction measures materialization against a client that decodes and recomputes; it is not a speedup over an optimized raw-scanning engine.

## 6.2 Same storage, different query (P1)

P1 stored the identical float16 envelopes in a pgvector `halfvec(410)` column [11]. Both tables occupied exactly 1,794,048 bytes, and all 1,826 envelopes were bit-identical, so envelope storage itself is no advantage. Zero-lag cosine search, however, shared only 64.7% of the lag-tolerant top ten and reproduced its order for 0.1% of queries. Scoring lags in the client on the same `halfvec` rows reproduced MorletDB's ranking exactly. In a later same-run comparison, median latency was 6.85 ms for MorletDB, 0.49 ms for zero-lag cosine and 20.3 ms for client lag scoring. The first two answer different questions, so their ratio is not a like-for-like speed comparison.

## 6.3 Scaling and approximate prefiltering (R2)

R2 asked how exact search scales and whether a standard approximate index can serve as its first stage [31]. We loaded 14 days of four CX stations from 2025, data no earlier study had read: 481 M samples in 128 gap-free recordings, appended in 451 s and stored at 6.2 bits per sample. Envelopes over 4096-sample windows at stride 2048 gave 234,762 windows (275 s to build). Galleries were nested by day, from 4,217 to 234,762 windows. The 100 queries were 50 random and 50 high-energy windows from the first day. Arm H-$K'$ took the top $K'$ windows by zero-lag cosine from a pgvector HNSW index ($m$ 16, `ef_construction` 64) on the same float16 envelopes and re-ranked them with the exact lag-tolerant distance in the database; arm Z used the cosine order alone. The pre-registered bar for the prefilter was recall@10 ≥ 0.95 at no more than a tenth of the exact p95 latency on the largest gallery.

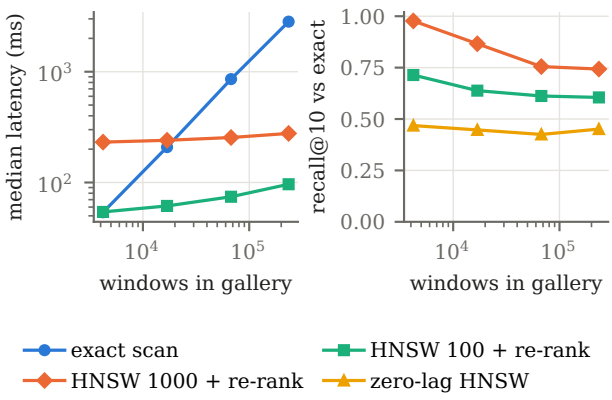



**Figure 3: R2 on fresh 2025 data. Left: median latency of exact lag-tolerant search and of HNSW-prefiltered re-ranking. Right: recall@10 of the approximate arms against the exact top ten. 100 queries per gallery; one client, warm cache.**

Exact search was linear at 12.1–12.8 µs per stored window at every size (Figure 3): 54 ms for 4,217 windows, 0.86 s for 67,472 and 2.8 s for 234,762. It stays interactive (under 100 ms) up to about 8,000 windows, which in practice means search filtered to one station-day or one machine.

The prefilter failed its bar. With $K'$=1000, recall@10 fell from 0.977 on the smallest gallery, where 1000 candidates are a quarter of it, to 0.743 on the largest, at 0.28 s; with $K'$=100 it fell to 0.61. Zero-lag cosine alone shared 43–47% of the exact top ten on these continuous recordings, below the 64.7% of P1's event windows. Random and high-energy queries behaved alike. The lag-tolerant neighbours of a window are thus not concentrated among its zero-lag neighbours, and a fixed candidate budget captures a shrinking share of them as the gallery grows. The index also costs more than the data: side table and HNSW took 516 MB, 1.4× the raw samples, while the envelopes themselves took 0.60× (more than R1b's 12.9% because half-overlapping windows store every sample in two envelopes).

Lag-tolerant search therefore needs either a filter that keeps galleries in the thousands, which the metadata predicates provide, or a shift-invariant first-stage key; an off-the-shelf vector index over the shifted representation is not one.

# 7 APPLICATION EVIDENCE AND NEGATIVE RESULTS

**Sleep staging by example (S1).** On 153 Sleep-EDF nights with a subject-disjoint split, staging each 30-second epoch by the 40 most similar expert-scored epochs from other subjects gave a mean per-subject Cohen's $\kappa$ of 0.583, against 0.541 for logistic regression on the same features (+0.042, 95% CI [0.016, 0.068]), with equal macro-F1 [32]. Retrieval thus exposes source-linked neighbours at no measured accuracy cost against this simple comparator; part of the

$\kappa$ difference comes from the regression's class-balanced weights. The result is far from state-of-the-art staging and makes no clinical claim.

**Morlet versus FFT.** In the same study, FFT band powers with the same band layout gave $\kappa$ 0.589 against 0.583 for Morlet energies. We found no advantage for the wavelet representation and made the extractor replaceable.

**Wavelet and progressive storage.** A compression benchmark found the tested wavelet decomposition codec 19–57% *larger* than scalar quantization with conventional coding at matched error bounds, and progressive raw storage missed its byte-saving gate (0.59 against $\leq 0.5$ of a cascade) because a separate envelope index dominated [33]. This motivated storing a retrieval representation *next to* losslessly coded raw data rather than a multiresolution raw format.

**Reference coding.** Coding a machining recording as a residual against an earlier execution of the same operation would have saved at most 0.0019% on the Bosch CNC data [24], because executions were too dissimilar sample by sample [34].

**Out of scope.** Three later studies tested retrieval-based forecasting of bearing and turbofan degradation and current-to-vibration estimation; none passed its gate [27]. They concern analytics on top of the store rather than storage and retrieval.

## 8 RELATED WORK

**Float compression.** Gorilla [3] and Chimp [4] XOR consecutive values and suit slowly changing metrics; ALP [1] finds decimal exponents and falls back to ALP-RD for high-precision doubles; FCBench [2] compares such methods across domains. Our results add a data class to this picture: calibrated sensor floats, for which a declared quantum and per-chunk step refinement beat decimal search, and for which exact replay of calibration arithmetic remains open.

**Waveform and audio coding.** Shorten and FLAC [5, 6] established polynomial or LPC prediction with Rice coding [7]; seismology stores counts with Steim compression in miniSEED [9]. MorletDB reuses these techniques and adds the float grid with exact corrections.

**Time-series and array stores.** TimescaleDB [10] compresses rows of samples in PostgreSQL, with Gorilla-style coding for floats. SciDB [15] and TileDB [16] manage dense arrays outside a relational engine. Columnar formats such as Parquet [14] and engines such as DuckDB [13] store sample columns well (Table 3) but leave recording semantics, replay-safe appends and interval read-back to the application.

**Similarity search.** FAST [17] fingerprints seismic waveforms for large-scale similarity search; the UCR suite [18] and matrix-profile methods [19] search subsequences under shift and warping. Graph indexes such as HNSW [12], available in PostgreSQL through pgvector [11], give approximate nearest neighbours under fixed metrics. MorletDB's contribution is not the measure but its placement, a lag-tolerant scorer inside the database next to the samples it links back to, and its measurement against these alternatives.

## 9 LIMITATIONS

**One machine, warm caches.** All timings come from one laptop-class machine, one client and warm caches; we did not measure disk-cold or concurrent workloads. R2 tested one HNSW configuration; larger graphs might raise recall, but not beyond the ranking disagreement that zero-lag search shows. Storage sizes are deterministic and transfer; latencies do not.

**Baselines.** TimescaleDB was tested in one configuration; ALP was measured inside DuckDB, including its block layout; FLAC is the only LPC-based integer codec. A codec that replays calibration arithmetic exactly would change the calibrated-float comparison.

**Data reuse and relevance.** P1 reuses R1b's data and the P2 inputs come from earlier studies; R1b, R2 and S1 used data no earlier study had read. Our main retrieval metric is fidelity (recall against an exact reference). Task relevance is supported only by a catalogue-distance proxy for seismic events and by sleep staging; neither establishes usefulness for industrial troubleshooting.

**Correctness.** Bitwise round trips on every measured chunk, replay tests and permission checks are engineering evidence, not formal verification of the codec, the transaction logic or access control.

## 10 CONCLUSION

Treating a recording, rather than a sample or a vector, as the unit of storage lets a relational engine keep original samples compact, keep search representations replaceable, and return results that read back exactly. On calibrated sensor floats, declaring the digitizer grid mattered more than codec sophistication: it turned a 60-bit fallback into 9–16 bits, and exact replay of calibration arithmetic is the remaining step to integer-codec size. For retrieval, an ordinary vector column stores envelopes equally well, but lag tolerance changes the answer, and a standard vector index over the same envelopes cannot stand in for the exact lag-tolerant scan, which is linear and interactive only on filtered galleries of thousands of windows. A shift-invariant index, and exact replay of calibration arithmetic, are the two open problems these results point to.

## ARTIFACT AVAILABILITY AND DISCLOSURE

Code (Apache-2.0), frozen protocols with their commits, harnesses, tests and raw result files are at https://github.com/merkhet-io/morletdb; datasets keep their own licences. This draft was prepared with AI assistance from the repository's reports; the author(s) must audit text, numbers and references before submission.

## REFERENCES

[1] Azim Afroozeh, Leonardo X. Kuffo, and Peter Boncz. 2023. ALP: Adaptive Lossless floating-Point Compression. *Proc. ACM Manag. Data* 1, 4, Article 230 (Dec. 2023), 26 pages. doi:10.1145/3626717.

[2] Xinyu Chen, Jiannan Tian, Ian Beaver, Cynthia Freeman, Yan Yan, Jianguo Wang, and Dingwen Tao. 2024. FCBench: Cross-Domain Benchmarking of Lossless Compression for Floating-Point Data. *Proc. VLDB Endow.* 17, 6 (2024), 1418–1431. doi:10.14778/3648160.3648180.

[3] Tuomas Pelkonen, Scott Franklin, Justin Teller, Paul Cavallaro, Qi Huang, Justin Meza, and Kaushik Veeraraghavan. 2015. Gorilla: A Fast, Scalable, In-Memory Time Series Database. *Proc. VLDB Endow.* 8, 12 (2015), 1816–1827. doi:10.14778/2824032.2824078.

[4] Panagiotis Liakos, Katia Papakonstantinopoulou, and Yannis Kotidis. 2022. Chimp: Efficient Lossless Floating Point Compression for Time Series Databases. *Proc. VLDB Endow.* 15, 11 (2022), 3058–3070. doi:10.14778/3551793.3551852.

[5] Tony Robinson. 1994. SHORTEN: Simple lossless and near-lossless waveform compression. Technical Report CUED/F-INFENG/TR.156, Cambridge University Engineering Department.

[6] Martijn van Beurden and Andrew Weaver. 2024. Free Lossless Audio Codec (FLAC). RFC 9639, RFC Editor. doi:10.17487/RFC9639.

[7] Robert F. Rice. 1979. Some Practical Universal Noiseless Coding Techniques. JPL Publication 79-22 (NASA-CR-158515), Jet Propulsion Laboratory.

[8] Bob Kemp, Alpo Värri, Agostinho C. Rosa, Kim D. Nielsen, and John Gade. 1992. A simple format for exchange of digitized polygraphic recordings. *Electroencephalography and Clinical Neurophysiology* 82, 5, 391–393. doi:10.1016/0013-4694(92)90009-7.

[9] International Federation of Digital Seismograph Networks. SEED Reference Manual, version 2.4. https://www.fdsn.org/publications/.

[10] Timescale, Inc. TimescaleDB documentation: hypertables and native compression. https://docs.timescale.com.

[11] pgvector contributors. pgvector: open-source vector similarity search for Postgres, version 0.8.0. https://github.com/pgvector/pgvector.

[12] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4, 824–836. doi:10.1109/TPAMI.2018.2889473.

[13] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proc. 2019 Int. Conf. on Management of Data (SIGMOD '19)*, 1981–1984. doi:10.1145/3299869.3320212.

[14] Apache Software Foundation. Apache Parquet format specification. https://parquet.apache.org/docs/.

[15] Michael Stonebraker, Paul Brown, Donghui Zhang, and Jacek Becla. 2013. SciDB: A Database Management System for Applications with Complex Analytics. *Computing in Science & Engineering* 15, 3, 54–62. doi:10.1109/MCSE.2013.19.

[16] Stavros Papadopoulos, Kushal Datta, Samuel Madden, and Timothy Mattson. 2016. The TileDB Array Data Storage Manager. *Proc. VLDB Endow.* 10, 4, 349–360. doi:10.14778/3025111.3025117.

[17] Clara E. Yoon, Ossian O'Reilly, Karianne J. Bergen, and Gregory C. Beroza. 2015. Earthquake detection through computationally efficient similarity search. *Science Advances* 1, 11, e1501057. doi:10.1126/sciadv.1501057.

[18] Thanawin Rakthanmanon, Bilson Campana, Abdullah Mueen, Gustavo Batista, Brandon Westover, Qiang Zhu, Jesin Zakaria, and Eamonn Keogh. 2012. Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping. In *Proc. 18th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD '12)*, 262–270. doi:10.1145/2339530.2339576.

[19] Chin-Chia Michael Yeh, Yan Zhu, Liudmila Ulanova, Nurjahan Begum, Yifei Ding, Hoang Anh Dau, Diego Furtado Silva, Abdullah Mueen, and Eamonn Keogh. 2016. Matrix Profile I: All Pairs Similarity Joins for Time Series: A Unifying View That Includes Motifs, Discords and Shapelets. In *IEEE 16th Int. Conf. on Data Mining (ICDM)*, 1317–1322. doi:10.1109/ICDM.2016.0179.

[20] Bob Kemp, Aeilko H. Zwinderman, Bert Tuk, Hilbert A. C. Kamphuisen, and Josefien J. L. Oberyé. 2000. Analysis of a sleep-dependent neuronal feedback loop: the slow-wave microcontinuity of the EEG. *IEEE Trans. Biomed. Eng.* 47, 9, 1185–1194. doi:10.1109/10.867928.

[21] Ary L. Goldberger, Luis A. N. Amaral, Leon Glass, Jeffrey M. Hausdorff, Plamen Ch. Ivanov, Roger G. Mark, Joseph E. Mietus, George B. Moody, Chung-Kang Peng, and H. Eugene Stanley. 2000. PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals. *Circulation* 101, 23, e215–e220. doi:10.1161/01.CIR.101.23.e215.

[22] Patrick Wagner, Nils Strodthoff, Ralf-Dieter Bousseljot, Dieter Kreiseler, Fatima I. Lunze, Wojciech Samek, and Tobias Schaeffter. 2020. PTB-XL, a large publicly available electrocardiography dataset. *Scientific Data* 7, Article 154. doi:10.1038/s41597-020-0495-6.

[23] GFZ German Research Centre for Geosciences and Institut des Sciences de l'Univers–Centre National de la Recherche CNRS-INSU. 2006. IPOC Seismic Network. Integrated Plate boundary Observatory Chile. Network code CX. doi:10.14470/PK615318.

[24] Mohamed-Ali Tnani, Michael Feil, and Klaus Diepold. 2022. Smart Data Collection System for Brownfield CNC Milling Machines: A New Benchmark Dataset for Data-Driven Machine Monitoring. *Procedia CIRP* 107, 131–136. doi:10.1016/j.procir.2022.04.022.

[25] Christian Lessmeier, James Kuria Kimotho, Detmar Zimmer, and Walter Sextro. 2016. Condition Monitoring of Bearing Damage in Electromechanical Drive Systems by Using Motor Current Signals of Electric Motors: A Benchmark Data Set for Data-Driven Classification. In *Proc. European Conf. of the PHM Society (PHME 2016)*. doi:10.36001/phme.2016.v3i1.1577.

[26] J. Lee, H. Qiu, G. Yu, J. Lin, and Rexnord Technical Services. 2007. Bearing Data Set. IMS, University of Cincinnati. NASA Prognostics Data Repository, NASA Ames Research Center. https://data.nasa.gov/dataset/ims-bearings.

[27] MorletDB contributors. 2026. MorletDB source repository, protocols and results. https://github.com/merkhet-io/morletdb.

[28] MorletDB contributors. Plain PostgreSQL baselines P1 and P2. `evaluation/plain_postgres/README.md` in [27].

[29] MorletDB contributors. B1: lossless storage against modern baselines. `evaluation/storage_baselines/` in [27].

[30] MorletDB contributors. R1b confirmation (2022). `evaluation/materialized/compact/results/conf-2022/RESULTS.md` in [27].

[31] MorletDB contributors. R2: retrieval scaling. `evaluation/retrieval_scaling/` in [27].

[32] MorletDB contributors. S1: sleep staging by retrieval. `evaluation/sleep/RESULTS.md` in [27].

[33] MorletDB contributors. Stage-1 compression benchmark. `evaluation/compression/results/ab70819/RESULTS.md` in [27].

[34] MorletDB contributors. X1: reference coding across repeated operations. `evaluation/compression/reference/RESULTS.md` in [27].

# A EVIDENCE LEDGER

| Claim | Source | Restriction |
|---|---|---|
| 6.12–11.53 bits/sample in DB | P2 | one TimescaleDB configuration |
| 24–35% larger (T int) | P2 | users must supply counts |
| ALP ≈60 bits on calibrated | B1 | DuckDB 2.0.0 implementation |
| 1.5–1.6× below zstd | B1 | 2× claim not supported |
| 1.3 / 6.7 bits of corrections | B1 | two calibrated inputs |
| 12.9% size, recall 0.9999 | R1b | fidelity, not relevance |
| 64.7% zero-lag overlap | P1 | different similarity semantics |
| 12 µs per window, linear | R2 | one machine, warm, one client |
| HNSW+re-rank recall 0.74 at 235 k | R2 | one HNSW configuration |
| $\kappa$ 0.583 vs 0.541 | S1 | one EEG channel; no clinical claim |